

From Request-Local KV Reuse to Finite-Cache Reuse Admission

Anonymous Artifact Draft

Abstract—Large language model serving stacks increasingly expose reusable prefix or KV state across requests, but converting that state into startup-latency reduction is not free. Realizing reusable state may require transfer, stitching, or format adaptation, so a runtime needs a decision at request arrival: should it fully reuse that state, partially materialize only a profitable segment, or recompute from scratch? This artifact studies that narrow three-action control surface. Its workload-driven offline path evaluates the arrival-time decision surface over named shared cases from llm-serving-workloads; its live path documents what the current out-of-tree vLLM seam actually realizes online. The resulting historical claim boundary is explicit: the runtime realizes native `full_reuse/recompute` and block-aligned segmented `partial_reuse`, while scheduler-owned counters distinguish requested, effective, and realized actions. In a preregistered 18-lifecycle online matrix, the controller improves over recompute but fails to beat the best fixed policy: mean matched TTFT regresses by 9.24% and 6.60% on the two workloads, and `always_full_reuse` wins all six matched rounds. We therefore report a negative result and fix the preregistered verdict as `stop_mechanism_direction`: the mechanism direction is closed without per-workload tuning or post-hoc workload search. Raw-bundle evidence covers 39 of the 141 completed lifecycles; the remaining 102 are aggregate-only supporting evidence, and this paper keeps that 39/102 reproducibility boundary. The negative result motivates a different, still-unproven question: whether full-reuse occupancy under finite paged-KV capacity creates a cross-request victim or SLO externality. We preregister a treatment-free fixed-policy crossover gate before any adaptive admission mechanism; this draft makes no claim that the crossover or treatment benefit has been observed.

Index Terms—LLM serving, KV cache, prefix reuse, materialization control, time to first token

I. Introduction

Recent LLM serving systems increasingly rely on prefix reuse, prompt caching, and reusable KV state to reduce prefill cost. However, a reusable prefix is not equivalent to a free latency reduction. In multi-GPU settings, reusable state may reside on another worker, may require transfer through interconnect or host memory, and may need partial rematerialization before decoding can start. As a result, the serving system faces a control problem: when a request arrives and some reusable state exists, how much of that state should be realized now?

Recent disaggregated and remote-KV serving efforts, including DistServe and Mooncake [1], [2], make this cost visible rather than hiding it behind a monolithic “cache hit” abstraction. Once reuse requires transport, re-layout, or partial reconstruction, the runtime needs a policy that

decides whether realizing the reusable state is worth the startup cost.

That decision is structurally closer to selective materialization than to a binary cache-hit classifier. A serving stack is not merely asking whether reuse exists; it is asking whether a reusable state fragment should be materialized now, how much of it should be materialized, and when recomputing the remainder is cheaper. This perspective is inspired by selective materialization work in other domains, where the key systems question is not “can this state be reused?” but “is materializing it worth the cost under the current workload and resource conditions?” [3].

This repository focuses on a narrow but practically important control point: request-arrival KV materialization. Instead of treating reuse as a binary choice, we study a three-action decision surface. The controller can fully materialize the reusable prefix, partially materialize only a profitable segment, or ignore the reusable state and recompute the prompt from scratch. This formulation is intentionally orthogonal to placement, admission, and online memory-evolution policy. Placement affects where reusable state resides; admission affects which work enters the system; memory evolution affects how state is written, retained, or retired over time. Our question begins after reusable state is already available and asks whether realizing it at request arrival is worth the cost.

The completed online evidence rejects that request-local controller as an optimization direction on the tested carrier. The post-negative question therefore changes the decision object rather than retuning it: can retaining locally useful full-prefix state impose a measurable eviction, residence, or deadline externality on concurrent requests under a fixed paged-KV budget? This possibility was not measured by the old experiment and remains gated by a treatment-free fixed-policy crossover study.

Historically, we hypothesized that request-varying re-computation and realization costs required an explicit three-action controller. M2 rejects that mechanism on the tested carrier: the controller loses to `always_full_reuse` in all six matched rounds. We therefore do not tune that controller or its thresholds further. The active hypothesis is instead a finite-cache externality question, and remains unobserved pending a treatment-free fixed-policy D0.

In contrast to a broad scheduler redesign, this paper targets one narrow and measurable hook. The benefit of that choice is methodological: it keeps the artifact honest about what is being optimized, and it makes offline and

online evidence easier to interpret.

II. Completed Historical Formulation

The following formulation records the closed mechanism evaluated by M2; it is not the current research direction. For each incoming request, the runtime observes a candidate reusable prefix together with signals about prefix length, estimated recomputation work, and estimated realization cost. The controller then chooses one of three actions:

- `full_reuse`: materialize the full reusable prefix,
- `partial_reuse`: materialize only a prefix segment and recompute the rest,
- `recompute`: ignore reusable state and run the prompt path from scratch.

This decision surface is deliberately narrow. It avoids conflating multiple control layers and makes the evidence interpretable: if the controller helps, the gain comes from better request-arrival materialization decisions rather than from a hidden change in scheduler, placement, or eviction behavior.

A. Signals and Objective

The controller reasons over a small signal set: reusable prefix length, estimated transfer latency, estimated recomputation latency, queue pressure, and a confidence estimate for whether the candidate reusable state is likely to produce useful saved work. The objective is to minimize startup latency, represented primarily by TTFT, while keeping realized KV traffic proportional to actual benefit.

We use these signals through a simple utility view. For a candidate action a , the controller estimates the startup benefit from avoided recomputation, then subtracts realization-side overheads such as transfer, stitching, and pressure amplification. Conceptually,

$$U(a) = B_{\text{recompute}}(a) - C_{\text{realize}}(a) - P_{\text{pressure}}(a).$$

The controller selects the action with the largest non-negative utility. Under this view, `full_reuse` corresponds to materializing the whole reusable prefix, `recompute` corresponds to rejecting materialization, and `partial_reuse` corresponds to selecting only a profitable prefix segment when early reuse has positive value but the tail does not.

This formulation is intentionally lightweight: it is not presented as a globally optimal estimator, but as the minimal abstraction needed to compare three actions on equal footing and to expose where runtime support is still missing.

B. Why Partial Reuse Matters

The partial action is the key difference between a binary reuse controller and a materialization controller. In practice, the early segment of a reusable prefix may be worth loading while the tail segment is not, especially when transfer cost grows faster than the remaining recomputation avoided. A binary policy cannot express that

breakpoint. This is exactly where the selective materialization perspective matters: the system should realize only the portion whose marginal utility remains positive, instead of treating “reuse” as an all-or-nothing event.

C. Current Partial-Reuse Boundary

The artifact keeps the boundary around `partial_reuse` explicit. In the offline decision study, `partial_reuse` is a first-class action: the controller may materialize only a profitable prefix segment and recompute the remainder. On the current live prefix-cache seam, `partial_reuse` is now realized only at block-aligned granularity: prefix-cache lookup is capped at the aligned reuse boundary, cache write-back is capped at the same boundary, and the recomputed tail is isolated under a request-scoped segmented hash namespace. This is stronger than the earlier fallback-only seam, but it is still not exact token-level materialization. Accordingly, the runtime keeps the observed decision separate from the effective action and records when a block-aligned `partial_reuse` cut point is realigned, dominated by `full_reuse`, or collapses to recompute.

III. Historical Design Perspective

The design goal is not to maximize reuse rate. The design goal is to minimize request latency subject to the actual cost of realizing reusable state. From this perspective, full reuse is only one candidate action, not the default answer. A partial action is useful when the early portion of a prefix has high reuse value but the tail has poor return once transfer and stitching overhead are considered. The recompute action remains necessary when realization cost dominates the saved prefill work.

This framing is important for systems evaluation. A policy that increases nominal reuse coverage but also increases transfer volume or startup delay is not an optimization. We therefore evaluate the controller with both latency and cost-oriented metrics, rather than reuse ratio alone.

This design perspective also sets the paper’s evidence boundary. A strong offline result means the decision rule is plausible under workload-grounded cost assumptions. It does not automatically imply an end-to-end serving gain until the same rule is tested through the live vLLM plugin path. Conversely, a live run that only observes decisions without enforcing them is useful for validating workload entry, prompt geometry, and service-side constraints, but it is not yet evidence that the controller changed runtime behavior.

IV. Completed Historical Methodology

The old controller study used two evaluation layers and kept them separate. This methodology is retained to explain the negative result, not as an imperative plan for further controller, seam, or threshold work.

The first layer is a workload-driven offline decision study over named shared cases from llm-serving-workloads. This

layer is used to compare the three-action decision surface across baselines, heuristic policies, oracle headroom, and cost-misestimation sensitivity variants.

The second layer is a live runtime-boundary path against a controlled Qwen2.5-7B-Instruct deployment. This layer is used to report which actions the current runtime seam realizes online and which actions still require fallback.

We compare fixed baselines that always recompute, always fully reuse, or apply a simple thresholded partial policy. We then evaluate an adaptive heuristic controller that estimates whether realization cost is justified. To bound the available policy headroom, we also report an oracle selector over the same three-action space. Finally, we include cost-misestimation sensitivity variants to test whether the heuristic remains stable when the runtime’s cost model is inaccurate.

These policy families answer different questions. The fixed baselines ask whether selective materialization is needed at all. The heuristic asks whether a small interpretable controller can improve on static rules once transfer cost, pressure, and confidence vary across requests. The oracle asks how much decision headroom would remain even if the runtime had perfect per-request knowledge within the same three-action space.

The primary metrics are mean and tail TTFT, recomputed prompt tokens, and KV bytes realized or transferred. Together these metrics distinguish beneficial reuse from expensive reuse.

A. Policy Families

The decision study compares five policy families.

Always recompute. This policy rejects all candidate reusable state. It is the control baseline for systems where realization overhead dominates and materialization is never worth the startup cost.

Always full reuse. This policy materializes the entire reusable prefix whenever one exists. It represents the opposite extreme and exposes how often naive “reuse whenever possible” logic overpays in transfer, stitching, or pressure cost.

Threshold partial policy. This policy is a static hand-tuned rule. It uses a fixed breakpoint to decide whether to fully reuse, partially reuse, or recompute. Its role is not to be optimal, but to provide a simple selective materialization baseline that is more expressive than binary reuse.

Adaptive heuristic. This was the main policy in the closed M2 study. It uses the signals in Section III-A to estimate whether full realization, partial realization, or recomputation has the highest utility for the arriving request. Unlike the threshold baseline, it can change its choice with queue pressure, reuse confidence, and estimated transfer-versus-recompute balance.

Oracle TTFT selector. This policy is offline only. It chooses the best action in hindsight using the same three-

action action space and is used only to estimate the remaining headroom above realizable heuristics. It is not presented as a deployable controller.

B. Sensitivity and Interpretation

The decision study also includes cost-misestimation variants that perturb the transfer-side and recompute-side estimates seen by the heuristic. These runs are important because the central question is not whether one specific cost model is perfectly calibrated; it is whether selective materialization remains useful when runtime estimates are noisy.

Under this methodology, the result interpretation is disciplined. If the heuristic consistently outperforms both static baselines, the evidence supports the need for an explicit materialization controller. If the oracle remains far ahead of the heuristic, then the paper’s remaining opportunity is primarily in better decision quality. If the offline headroom is large but the live path is still weak, then the bottleneck is runtime realization rather than policy value.

C. Workload Source of Truth

The study deliberately enters through the sibling llm-serving-workloads repository rather than a repo-local toy catalog. This keeps workload construction policy-neutral and prevents the paper from baking a one-off benchmark family directly into the optimization artifact.

D. Baselines

The baseline set is intentionally simple and interpretable: always recompute, always full reuse, and a fixed threshold-based partial policy. The adaptive heuristic is then compared against these baselines, while an oracle selector is used only to estimate remaining headroom within the same three-action space.

This baseline structure mirrors the paper’s decision-centric scope. The goal is not to claim a sophisticated scheduler or memory manager, but to ask whether a runtime can outperform static materialization rules once realization cost is made explicit.

E. Historical Research Questions (Closed)

The completed M2 methodology was organized around three concrete questions:

- RQ1: Do workload-grounded request-arrival traces actually exhibit a non-trivial three-action decision region, or do static “always recompute” and “always full reuse” rules already suffice?
- RQ2: When a three-action region exists, can a small interpretable heuristic recover a meaningful fraction of oracle headroom?
- RQ3: If the offline decision value exists but the live runtime still falls back, is the dominant remaining gap policy quality or runtime support for exact segmented materialization?

M2 closes these questions as an optimization route on the tested carrier. They are retained only to explain what was tested and why the negative result rules out more threshold tuning or seam continuation under the same protocol.

F. Active Research Questions and D0 Methodology

The active study asks three different questions. RQ1: under fixed paged-KV capacity, does always fully reusing locally available state create a reproducible victim-residency, queue-wait, or deadline externality for concurrent requests? RQ2: does the winner switch across pre-registered low/high overlap and roomy/constrained cells when comparing only deployable fixed policies? RQ3: can native block, victim, residence, wait, action, completion-identity, and cleanup receipts causally explain that switch?

D0 contains no adaptive treatment. It matches the request set, order, model, runtime, concurrency, and KV-block cap across always_full_reuse, always_recompute, the frozen old controller, fixed capacity/concurrency gates, and the unmodified native cache policy. A future-aware oracle is an offline upper bound only. The finite-cache externality remains unobserved until this fixed-policy crossover passes its correctness, provenance, performance, and stop gates. Implementation and experiments remain the student owner’s responsibility.

G. Current Evidence Scope

The repository currently supports a catalog-driven offline decision matrix that follows the shared benchmark case order exported by llm-serving-workloads. This lets the paper evaluate the same default workload set consumed by sibling artifacts instead of carrying a second hardcoded subset locally. It also supports a live runtime-boundary path that enters through the same shared workload repository and runs against a controlled Qwen2.5-7B-Instruct endpoint.

The claim boundary is therefore explicit. The offline layer supports claims about the three-action arrival-time decision surface. The live layer supports claims about runtime realization status on the current seam: native support for full_reuse and recompute, plus block-aligned segmented support for partial_reuse with explicit effective-action and fallback reporting.

V. Live Runtime Boundary

The live path is intentionally narrower than a broad online benchmark suite. It is used to answer one question: what does the current runtime seam really do?

On the current out-of-tree prefix-cache path, full_reuse is realized through stable anchor-scoped cache domains, and recompute is realized through request-scoped cache bypass. partial_reuse is realized only at a block-aligned segmented boundary on this path: the runtime caps lookup and shared-cache commit at the aligned reuse boundary and resets the tail hash chain into a request-scoped

TABLE I
Representative live runtime-boundary matrix.

Workload	Setting	Mean (ms)	p95 (ms)	RPS
Knowledge	old baseline	3289.302	3641.928	1.209
Knowledge	old tuned	3860.692	4170.369	1.031
Knowledge	new baseline	3054.712	3383.972	1.303
Knowledge	new tuned	2980.648	3214.822	1.335
Tool scaffold	old baseline	8856.437	10627.872	0.896
Tool scaffold	old tuned	11068.224	12187.492	0.717
Tool scaffold	new baseline	9362.399	10302.383	0.848
Tool scaffold	new tuned	9186.471	10182.569	0.865

TABLE II
Additional live boundary cases on the segmented seam, including the tokenizer-faithful dynamic-RAG rerun.

Workload	Completed	Mean (ms)	p95 (ms)
Shared-prefix assistant	40/40	10469.485	16098.330
Session maintenance	40/40	6120.100	6488.292
Dynamic RAG update	32/32	2634.091	3073.478

segmented namespace. This seam is still narrower than exact token-level segmented materialization, so the live path must continue to report observed versus effective action explicitly.

A. Fallback Taxonomy

This distinction is essential for paper truthfulness. The repository has a real online control path for all three actions, but partial_reuse is only realized on a block-aligned segmented seam rather than as exact token-level materialization. The runtime therefore exposes a fallback taxonomy rather than hiding the remaining gap. When the policy selects partial_reuse, the live record includes the observed decision, the effective action, the support tier, and the fallback reason stating whether the cut point was realigned to runtime blocks, dominated by full_reuse, or collapsed to recompute.

B. Representative Live Matrix

To test whether the recent online improvement comes from the runtime seam or only from changing the decision mix, we compare four controlled live conditions on two representative workloads: old seam plus baseline knobs (floor=256, confidence_penalty=4.0), old seam plus tuned knobs (floor=128, confidence_penalty=8.0), new segmented seam plus baseline knobs, and new segmented seam plus tuned knobs. All runs use the same Qwen2.5-7B-Instruct deployment and enter through the shared workload catalog.

This matrix is a descriptive runtime-boundary result, not a stable performance gain. The completed M1 analysis uses three independent restarts as temporally blocked lifecycle units. Its seam main effects on the knowledge and tool workloads are close to zero, and the unchanged protocol stops at the registered 5% minimum meaningful effect rather than adding repetitions. The safe claim is therefore that the segmented seam realizes and accounts

for partial reuse; the matrix does not establish a general seam benefit or a win over fixed policies.

Table II extends that statement beyond the two four-setting comparison workloads. The shared-prefix assistant and session maintenance cases show that the segmented seam can sustain stable online execution on prefix-rich and long-continuation workloads without reopening the offline-only evidence boundary. The dynamic-RAG rerun is methodologically more important: the earlier failed run was not evidence that this case inherently exceeded the Qwen2.5-7B context window. Instead, the failure came from constructing the live workload with a whitespace tokenizer, which distorted the shared-workload token budget relative to the tokenizer used by the actual endpoint. Once the live runner was switched to the real Qwen tokenizer, the same `dynamic_rag_corpus_update` case completed cleanly and showed the expected action transition: early turns remained in the block-aligned `partial_reuse` regime, while later turns moved to effective `full_reuse` as retrieval overlap increased. This correction matters for paper truthfulness because it turns a misleading “context-window boundary” story into the real systems conclusion: live evidence depends not only on the runtime seam, but also on whether workload construction is faithful to the serving tokenizer.

C. Observed and Effective Actions

The new segmented-baseline run also clarifies why the claim must remain tied to runtime realization rather than nominal policy choice alone. On `shared_scenario_multi_turn_knowledge_service`, the workload splits into 8 effective recompute requests and 24 effective `partial_reuse` requests. On `shared_tool_scaffold_agent`, all 64 requests are observed as `partial_reuse`, but the effective runtime actions become 48 block-aligned `partial_reuse` requests and 16 `full_reuse` requests after re-ranking. This behavior shows that the seam and the guardrails jointly determine the realized action mix: the controller cannot be evaluated honestly by observed decisions alone.

D. M2 Fixed-Policy Benefit Boundary

The runtime-realization result above does not establish that a controller beats a fixed policy. M2 closes that gap with a preregistered comparison of `always_recompute`, `always_full_reuse`, and the current three-action controller. The study uses two existing workloads, three temporally blocked rounds per workload, a balanced first-position order, and a fresh service lifecycle for every cell. All 18 graph-mode bundles pass raw validation, covering 576 successful requests. The statistical unit is the lifecycle, not the individual request.

Table III shows why nominal action counts are insufficient. On tool scaffold, the controller’s median applied mix is 8 recompute, 8 partial, and 16 full actions, but the engine realizes 16 recompute, 16 partial, and zero full requests. It avoids 4096 prefill to-

kens while recomputing 4192 realized tail tokens. On knowledge service, all 32 requested actions are partial; runtime re-ranking yields 16 effective partial and 16 effective full actions, while the engine realizes 8 recompute and 24 partial requests. The median lifecycle avoids 7168 prefill tokens but recomputes 9720 tail tokens. Lookup, cache commit, boundary alignment, and tail isolation are all included in the measured online path. The realign column corresponds to `partial_reuse_cut_point_realigned_to_runtime_hash_blocks`; To full corresponds to `block_aligned_partial_reuse_dominated_by_full_reuse`. No M2 controller lifecycle records a `block_aligned_partial_reuse_collapses_to_recompute` fallback.

The benefit boundary is negative. Relative to the best fixed policy in each matched round, controller TTFT regresses by 9.24% on tool scaffold and 6.60% on knowledge service. End-to-end latency changes are only +0.16% and +0.66%, and throughput changes are -0.16% and -0.68%, so neither compensates for the TTFT regression. `always_full_reuse` is the online winner in all six matched rounds. The cost model predicts the same winner in all six rounds; there is therefore no evidence that a physically meaningful global parameter calibration would reverse the result. Following the stopping rule, we record the negative result and do not tune the controller separately per workload.

VI. Offline Decision Study

The offline study remains valuable because it is the only place where the full three-action decision surface is currently evaluated uniformly across all selected workload cases. It is the right layer for reasoning about whether `partial_reuse` has a meaningful decision region and how that region changes across workload families.

Equally importantly, the offline study is the layer where selective materialization can be evaluated without overstating runtime support. It lets us ask whether a profitable partial region exists at all, how wide that region is under cost-model error, and how much headroom would remain even if the live runtime still lacks exact segmented materialization.

Figure 1 uses the workload-family labels preserved by the historical offline artifact. The figure is retained as historical decision-study evidence and is not promoted to a new run.

The resulting picture is also less flattering than a simple “selective-materialization is obviously beneficial” story. Most workloads still lean strongly toward `full_reuse`. Among the five core policies, `always_full_reuse` remains the best mean-TTFT policy on 18 of the 25 shared-catalog workloads. However, once the offline evaluator stops treating `partial_reuse` as a fixed-fraction proxy and instead optimizes a confidence-aware cut point, the action no longer disappears entirely. The oracle now selects `partial_reuse` on 71 of 1296 requests, and the adaptive heuristic selects it on 365 requests, concentrated in a

TABLE III

Median per-lifecycle M2 controller mechanism decomposition. Avoided prefill and recomputed-tail values come from engine counters, not requested actions. Performance columns are mean matched deltas against the best fixed policy; positive latency deltas and negative RPS deltas are regressions. Peak cache is the maximum runtime cache-usage fraction. R/P/F denotes recompute/partial/full. Realign counts block-boundary alignment; To full and To recompute count the corresponding dominance and collapse fallback reasons.

Workload	Avoided	Tail	Lookup	Align	Commit	Isolation	TTFT Δ	E2E Δ	RPS Δ	Peak cache	Boundary
Tool scaffold	4096	4192	0.819	1.530	67.685	0.116	+9.24%	+0.16%	-0.16%	0.449%	fallback_or_no_gain
Knowledge service	7168	9720	1.010	2.037	39.060	0.225	+6.60%	+0.66%	-0.68%	0.524%	fallback_or_no_gain
Workload	Observed R/P/F		Effective R/P/F		Realized R/P/F		Realign	To full	To recompute		
Tool scaffold	8/24/0		8/8/16		16/16/0		8	16	0		
Knowledge service	0/32/0		0/16/16		8/24/0		16	16	0		

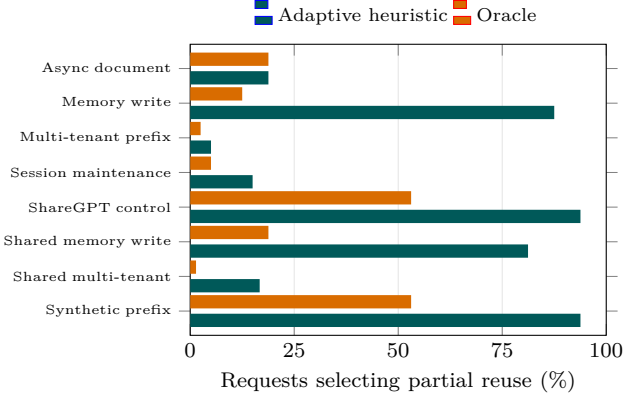


Fig. 1. The eight workload cases in which the historical offline oracle selects partial reuse at least once. Partial reuse is concentrated rather than universal; the heuristic also over-selects it in several cases.

small set of memory-write and session-affine cases. The static threshold policy still pushes too many workloads into partial_reuse and remains clearly worse overall, while the adaptive heuristic recovers only a limited slice of that new region.

That result identifies a model-side control region, but M2 shows that it does not translate into an online win over the best fixed policy on the tested workloads. The distinction is decisive: offline oracle headroom and realized partial requests are not evidence of controller benefit. The current artifact therefore supports a negative boundary result, not a claim that the adaptive heuristic recovers useful online value.

VII. Related Work Perspective

Two threads are most relevant to this artifact.

First, recent LLM-serving systems such as DistServe and Mooncake make reuse realization costs explicit by exposing remote or disaggregated state movement instead of hiding all reuse behind a local cache abstraction [1], [2]. These systems motivate measuring realization cost and finite-capacity externalities explicitly. They do not rescue the rejected request-local controller: reusable KV state may save computation, but retaining or realizing it can also consume bandwidth, displace blocks, increase queue wait, or amplify memory pressure.

Second, this artifact borrows its decision perspective from selective materialization work outside LLM serving.

LEAP frames materialization as a cost-sensitive online choice rather than a static yes-or-no optimization target [3]. We borrow that viewpoint, not the domain mechanism. Our problem is not video-view sampling, but arrival-time KV-state realization with a three-action space of full_reuse, partial_reuse, and recompute. The connection explains the historical controller formulation. The current contribution is the negative mechanism boundary plus a falsifiable fixed-policy test for a distinct finite-cache externality, not a claim that a new controller already exists.

VIII. Artifact Boundary

This repository intentionally separates decision-side evidence from live runtime realization evidence. We do not present offline oracle results as deployed gains, and we do not treat the existence of reusable state as proof that materialization is beneficial. We also do not reframe this repository as a general admission or memory-evolution artifact simply because some workloads contain broader state semantics.

A. Reproducibility Boundary

The M2 raw bundles cover 39 of the 141 completed lifecycles and are archived both in the repository and in release m2-issue3-closure-evidence-20260811 (m2_raw_bundles_39_lifecycles.tar.gz), so those 39 lifecycles can be reviewed request-by-request. The remaining 102 lifecycles are aggregate-only supporting evidence: their raw bundles originally lived in /tmp on the execution machine and are no longer retrievable. Per-request review claims therefore stay at the 39/102 boundary until those bundles are archived to a persistent, content-addressed location.

IX. Current Contributions

- 1) A preregistered negative result that closes the request-local three-action controller and its threshold/seam continuation: it loses to always_full_reuse in all six matched M2 rounds.
- 2) An explicit evidence boundary separating offline, live-runtime, replay, host-fixture, simulation, projected, and derived evidence, including the 39-lifecycle raw-bundle versus 102-lifecycle aggregate-only provenance limit.

- 3) A new, falsifiable finite-cache externality question with mandatory strong fixed baselines, native causal receipts, correctness oracle, performance gate, hard stops, and a two-week D0 delivery contract.
- 4) A treatment-free D0 admission rule: no adaptive mechanism is admitted unless a preregistered fixed-policy winner switch is observed and explained.

- [3] X. Yan et al., “Predictive and near-optimal sampling for view materialization in video databases,” in Proceedings of the 2024 International Conference on Management of Data, 2024.

X. Limitations and Direction Decision

The study covers two preregistered, mechanism-relevant workloads and three temporally blocked lifecycle rounds; it is descriptive rather than a claim of universal dominance. Exact token-level partial materialization remains outside the block-aligned seam, and broader semantic candidate generation belongs to a separate project boundary.

Nevertheless, the result is sufficient for the decision registered here. Both online workloads lose to the same fixed policy, the model agrees with that ordering, and the controller’s additional realized work explains the gap. We therefore fix the verdict as `stop_mechanism_direction` in the final summary and this paper boundary: the current arrival-time three-action mechanism direction is closed as a negative result, and we do not continue submission-oriented tuning of this controller. The conclusion is bounded to the matched model/runtime/catalog tested here, not a universal impossibility theorem. Reopening the question would require a materially different mechanism or workload semantics, not another sweep of the present cost knobs.

A. Post-negative finite-cache contract

The only admitted `reframe` first compares `always_full_reuse`, `always_recompute`, the frozen old controller, fixed concurrency and KV-block caps, and the unmodified native cache policy under matched low/high overlap and roomy/constrained capacity. A future-aware oracle is an offline upper bound only. No treatment is implemented in D0.

D0 requires `always_full_reuse` to retain at least one low-pressure win while another fixed deployable arm wins at least two preregistered constrained-overlap cells. In those cells, full reuse must have at least 10% worse p95 TTFT or at least 5% lower SLO goodput, explained by native block, victim, residence, wait, action, and cleanup receipts with 100% correctness. Otherwise the `reframe` stops. Only after D0 passes may a held-out M1 evaluate a marginal-externality admission rule; M1 requires at least 5% SLO-goodput gain, no more than 5% aggregate p95 regression, and zero correctness or cleanup errors. Full details, evidence labels, hard stops, and the two-week delivery are frozen in `docs/FINITE_CACHE_REUSE_ADMISSION_CONTRACT.md`.

References

- [1] “Distserve,” 2024.
- [2] “Mooncake,” 2025.